

Department of Engineering Science
Faculty of Engineering

UNIVERSITY OF AUCKLAND

Historical Data Prediction for Geothermal Systems using Data-Driven Modelling

Daniel Clark

Supervisors:

Dr Michael Gravatt
Associate Professor Andreas W. Kempa-Liehr

Partner:

Michael Terekhin

August 5, 2024

Contents

1	Introduction	1
1.1	Problem Background	1
1.2	Available Data	1
2	Related Work	4
2.1	Bayesian Modelling	4
2.2	Machine Learning Approaches	5
2.3	Physics Informed Machine Learning	6
2.4	Solving as an Optimisation Problem	6
3	Methodologies	7
3.1	Project Scope	7
3.2	Variable Definition	7
3.3	Work Strategy	8
3.4	Tools	8
4	Mathematical Formulation	10
4.1	Formulating an Initial Model	10
4.2	Solving The Model Analytically	11
4.3	Relaxing Model Assumptions	13
4.4	Applying Tikhonov Regularisation to Reduce Overfitting	13
4.5	Solving Least Squares and Tuning Parameters	15
4.6	Quantifying Model Uncertainty	18
5	Results	19
5.1	Final Results	19
5.2	Comparison with other models	19
5.3	Conclusions	19

Chapter 1

Introduction

1.1 Problem Background

Geothermal energy is a renewable and environmentally-friendly energy source which can last for thousands of years [1]. Geothermal reservoirs hold water (in liquid and gaseous forms) at high temperature and pressure in large underground volumes of porous rock. This hot steam can be extracted to provide physical heat to a town or wider area, or it can be used to generate electrical energy by driving large steam turbines [2]. The electricity generated by this can then be fed into the electrical grid, which both provides the country with a clean source of renewable electricity generation, and can earn profit for whoever is running the geothermal site.

New Zealand's North Island is fortunate to have an abundance of Geothermal reservoirs. The majority of these lie in the Taupō Volcanic Zone, which runs Southwest from Whakaari (White Island) through Lake Taupō, to Mount Ruapehu, as shown in Figure 1.1. All of New Zealand's available geothermal resources have been identified, and we are not able to create any more, so we must be sure that we are using these geothermal resources responsibly. Because of the numerous benefits of using geothermal electricity generation, it is important to invest in geothermal energy, and ensure that we are properly caring for our geothermal resources. If too much mass is extracted from a geothermal well, then it may drastically reduce the amount of time the geothermal well can operate for. It is important to not extract too much mass from a geothermal reservoir to maximise its active lifespan [4]. To determine whether we are taking 'too much', we can use reservoir models to monitor the health of our geothermal reservoirs. Geothermal reservoir modelling is also used to predict the future of geothermal fields. It is important that these models are accurate because the models created can be used to make important decisions regarding the continued operation of the wells, and can act as strong evidence of profitability, driving economic investment in the field.

1.2 Available Data

Michael Gravatt, PhD and John O'Sullivan, PhD have been constructing geothermal reservoir models to predict the future health of geothermal well sites. Gravatt and O'Sullivan are two experts in the field of reservoir modelling from the Geothermal Institute at the University of Auckland. The Geothermal Institute [5] is the University of Auckland's hub of geothermal science, engineering, and geothermal reservoir modelling. Before Gravatt and O'Sullivan can run their predictive models they first have to inspect the available data, because the accuracy of a reservoir model depends highly on the accuracy and quantity of the measured data that it is fed. Often, the only frequently measured data is 'total steam flow' which is measured at a separator. Separators are normally fed by multiple wells, so it is often unclear how much of this steam has come from each of the wells that contribute to the separator.

Geothermal sites tend to measure data that is most important to their day-to-day operations

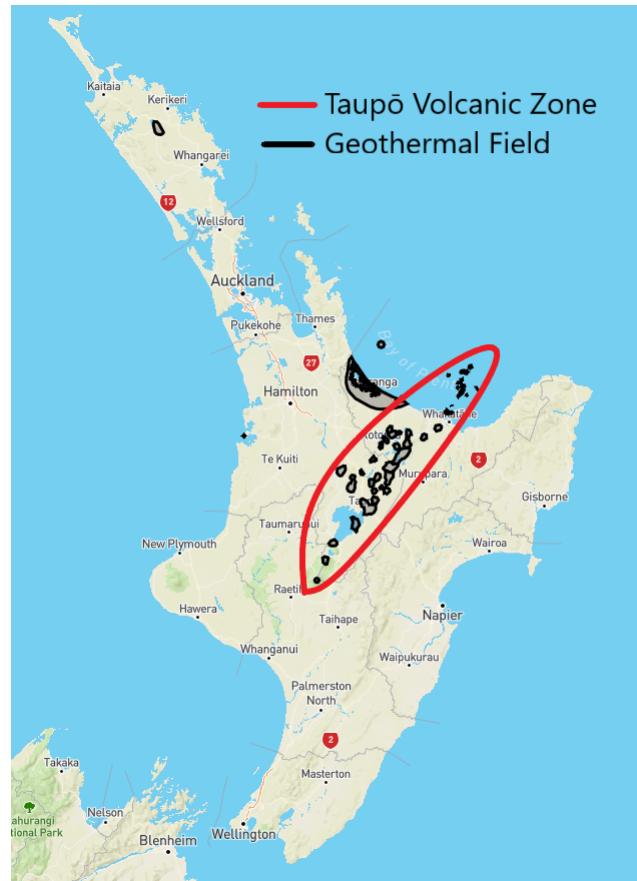


Figure 1.1: Geothermal fields and Taupō Volcanic Zone in New Zealand's North Island. Adapted from [3].

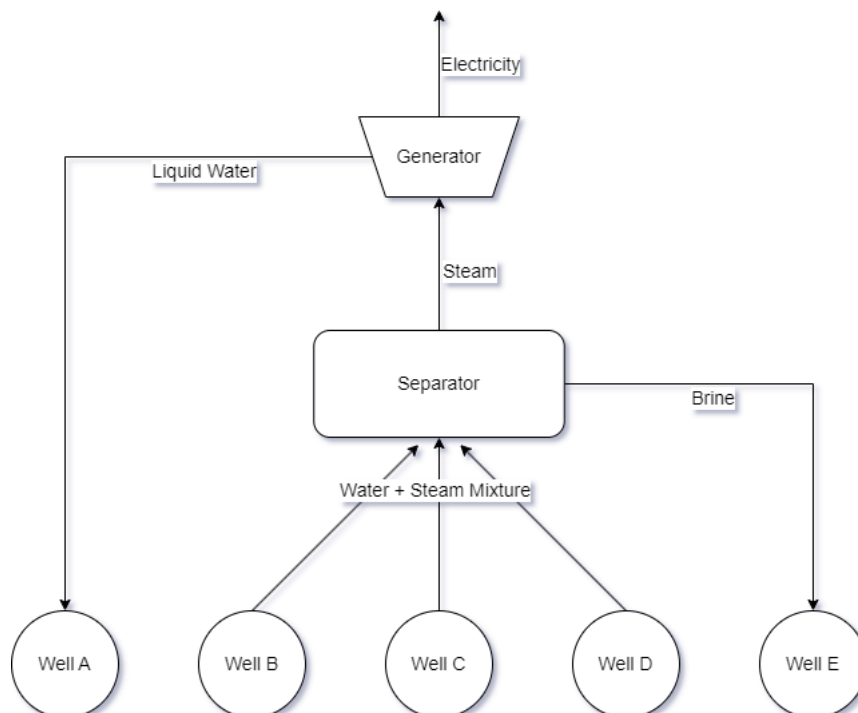


Figure 1.2: Simplified example of power generation process at geothermal field.

most frequently. For example, steam-flow data will be measured often, because these measurements will directly impact how much electricity they will be able to feed into the electrical grid. This is usually measured at the generator, which is normally powered by multiple geothermal wells. Measurements can be taken at different points during the process of geothermal power generation. Figure 1.2 shows that these measurements taken at the generator or separator are not representative of a single well, but rather of a number of wells, which are each contributing an unknown amount of mass to the separator. These wells will extract both liquid water and gaseous vapour from underground reservoirs, which are separated in a separator. The liquid water component of the extracted mass is generally re-injected as ‘brine’ into another well, while the extracted steam continues to the generator, where it generates electricity.

To determine how much mass is being extracted from an individual well, as required for effective reservoir modelling, we must use measurements that were taken at the well itself. These are called Total Flow Test (TFT) measurements, and are only taken occasionally for any given well, as the process of taking these measurements is difficult, and may require turning the well off for a short time. Because TFT measurements are taken so occasionally, often the available data for reservoir modelling is very limited. Using a combination of TFT measurements and total steam flow measurements, it is possible to predict the extraction history of a given well, however it is difficult and time-consuming to do this manually.

Currently, Gravatt and O’Sullivan are forced to predict the extraction history of a well manually, before they can begin reservoir modelling for that specific well. Manually inferring dense data from sparse measurements is an iterative process that follows the general structure of an educated ‘guess and check’. To do this, they begin by estimating the dryness and average contribution of each well that is contributing to the separator, by inspecting the relevant TFT measurements. After this, they calculate what total steam flow would result from these estimates, and compare this to the measured data. They then note any discrepancies, and try to change their estimate to fix these, repeating until the estimate matches the measured data well. However, this is not a very efficient or accurate method. This method assumes that values like dryness and contribution for each well remain constant over time, which is not a realistic assumption. Sometimes, they try to account for this by changing these values over time, but this makes the process more difficult and the results remain uncertain. There is a need to improve this process by automating the historical data prediction, which is why this project was created.

Myself and Michael Terekhin, my project partner, will use data-driven modelling methods to automate this process. Alongside supervision from Michael Gravatt, a geothermal reservoir modelling expert and Andreas Kempa-Liehr, who is an expert in machine learning and Bayesian modelling, we will model this situation in a number of ways, moving forward with the methods that prove most useful. This report will begin by noting relevant research from related works, and investigating which data-driven modeling approaches could work best for our unique situation. Then we will outline our process, highlighting methods which work poorly or well, and show how we iteratively improve our model before presenting our final data-driven model(s).

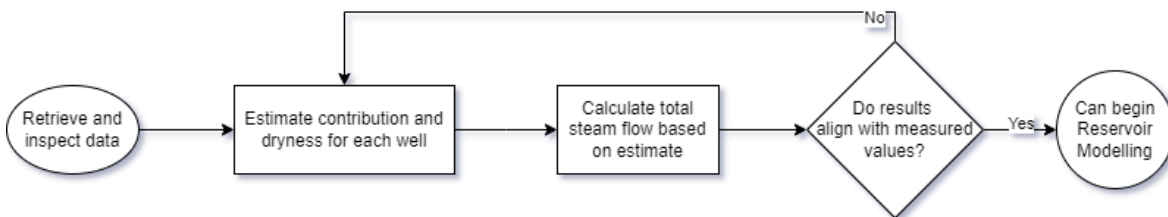


Figure 1.3: Simplified flowchart of manual process for predicting historical geothermal data.

Chapter 2

Related Work

Geothermal reservoir modelling is a large field where many approaches have been used to model geothermal wells. There are three classifications of geothermal reservoirs: high temperature above 225 degrees Celsius, low temperature below 125 degrees Celsius, and intermediate temperature between [6]. Higher reservoir temperatures lead to higher enthalpy of extracted fluid, and mean more electrical energy can be generated. This why it is important to use reservoir modelling to model the pressure and temperature of a geothermal reservoir over time. It gives the ability to predict the future temperature and pressure of a given reservoir, so that business decisions can be made to maximise profitability. Reservoir modelling can also ensure that we do not extract too much liquid from the reservoir, so as to keep the reservoir stable and healthy. Reservoir modelling generally models only a single well/reservoir within a larger geothermal field, so this is why it is important to predict the geothermal history of individual wells from total steam flow data, as we are doing. Below, we will discuss a number of techniques that could be used for predicting the required data, and discuss the advantages these techniques can provide. In Methodologies, we will begin using these methods to generate data-driven models.

2.1 Bayesian Modelling

Bayesian Modelling is a statistical method where prior distributions based on existing contextual knowledge are used to generate posterior distributions, which can be used to generate predictions and make analyses [7]. While Bayesian modelling has been applied to a geothermal context before [8], it has not been directly applied to the data prediction scenario described above. The most similar existing research largely comes from unrelated fields, where a similar allocation problem was encountered, such as the ‘Bellybutton Problem’, and the ‘Lions, Tigers and Bears’ discussed in the book ‘Think Bayes’ [9]. This suggests that, when undertaking Bayesian modelling of these situation, when it is known that all variables must sum to one, that a Dirichlet distribution is the appropriate prior distribution.

One approach within the Bayesian framework is Monte Carlo simulation. This is a method involving repeatedly generating random samples taken from the prior distribution, and then selecting the samples that go ‘near’ our data points, as feasible solutions, which are then used to generate the posterior distributions [10]. This can help us to quantify the uncertainty [10] of our estimates through results such as prediction intervals. However, this can lead to overfitting of uncertain and potentially noisy data. To prevent overfitting, we can use ensemble methods. These are methods such as Bayesian Voting, that have many models simultaneously and then take a weighted average of all the models’ results to give a final result [11]. This method of combining multiple ‘weak’ regressors into one ensemble model helps to reduce overfitting, because each individual model is trained on only a subset of the training data. This means they are less fit (they may even be underfitting) as individuals, but as a whole come together to give good results.

When engaging in traditional Bayesian modelling and computation, it is important to

know the relevant likelihood function or at least be able to estimate it by computing it numerically. In some situations this is not available and this is when Approximate Bayesian Computation (ABC) can be used to sidestep the likelihood function [12, 13]. ABC is a likelihood free inference method where samples are generated from the prior distribution, and run through the model to examine their similarity to the posterior distribution, using summary statistics, such as the mean, variance, range, maximum [14], although the types of summary statistics that can be used are highly context-dependent. Using a metric such as Euclidean distance, we measure if these statistics are ‘close’ enough to those of the posterior, and if so, accept that sample. As this is an ensemble method [13], this is then repeated to generate many samples. This makes it resistant to overfitting so could be a good method to use. The choice of summary statistics is important and often requires domain knowledge to choose appropriate statistics [15], however the process can be more computationally efficient than other Bayesian methods due to the summary statistics reducing the dimensionality of our model.

Gaussian Process Regression (GPR) is a Bayesian machine learning method that involves using a kernel to define a covariance matrix between variables. GPR can only be used when certain assumptions are met, including that the dataset, and the associated noise within, follow Gaussian distributions [16]. While our dataset cannot have perfectly Gaussian distributions, as the physical requirement that all contributions at a given time sum to one ensures that samples are not independent. Also the lower bound of 0 and upper bound of 1 ensure that a true Gaussian distribution is not appropriate, however, the Dirichlet distribution as mentioned above acts locally in a similar way to a Gaussian Distribution, so it is worth investigating whether this method can be applied to our dataset.

2.2 Machine Learning Approaches

Machine learning (ML) is the process of creating algorithms that learn how to improve themselves from their experiences [17]. In practice, a machine learning model is a model that will get better with the more data you train it on. There are many types of learning algorithms, some of which we will discuss below. The general goal of a machine learning algorithm is to find a model that minimises a chosen loss function [18]. When each additional piece of training data is fed into the algorithm, it will work out how to use the additional information to keep minimising the loss function, getting better and better until it runs out of training data. However, this process can lead to overfitting, which is when a model learns too effectively on the training data to the point that it captures the noise within the training data, instead of just capturing the desired real trend. To avoid overfitting, we often add a regularisation term onto our loss function, which can aim to either decrease the number of non-zero parameters in our model (Lasso regression), or to decrease the magnitude of parameters in our model (Ridge regression), or a combination of both.

Following the lead of the oil and gas industries, the geothermal industry’s use of machine learning has been increasingly exponentially since 2018 [19]. In a geothermal context it has previously been used in the context of well exploration, heat flux prediction, reservoir modelling, and above-ground component modelling, but not been used previously in the exact context we are looking to use it in [20]. We believe that it could be effectively applied in this area to automate the historical data prediction process.

Random forest regressors use an ensemble of decision tree regressors to predict the desired output. The decision trees each involve a sequence of splits from the root node down to the leaf nodes, where the leaf node will give the input data a predicted output. Each of these decision tree regressors can be given a maximum depth and complexity, and the large number of trees effectively being ‘averaged’ make it very difficult to overfit a random forest model [21]. Random forests also require relatively few computations, therefore are a very appealing option for early machine learning models.

Neural Network machine learning models often need to be provided large amounts of training data [22] to become effective predictive models. This may be an issue in our

context because the amount of data we could provide it is limited by the data which has been measured at the geothermal site. Keeping this in mind, this limits the sorts of models which we will attempt to use. For example, deep learning methods often require large amounts of data to become more effective than simpler models [23].

However, we may be able to sidestep this issue by generating additional simulated training (and testing) data to feed our models. Gravatt and O’Sullivan have created existing simulations that could generate the data required. Also, from one simulation, we can exclude and include different pieces of data to effectively create multiple pieces of training data from just the one simulation. So while we may not be able to feed much data into our model to generate predictions, our model will have a lot of data to learn from, so could still generate helpful results. If we use a (deep) neural network we should be able to extract all possible information from the limited data provided and has a chance to model our situation effectively. We can also use data augmentation to generate additional pieces of data [22] if our NN still did not have enough data to give effective results. Data augmentation involves creating copies of training data that are similar, but have been modified in some realistic way. For example, in our context, to augment training data, we could choose a different subset of TFT measurements to be provided to our model, such that it appears as a completely new piece of training data from the model’s perspective. We could also augment data by artificially adding noise (small perturbations) to our dataset.

2.3 Physics Informed Machine Learning

The issue with the above machine learning methods is that the contribution outputs for each well are not guaranteed to sum to one. This means that while our predictions may look accurate, they will be physically impossible. To fix this issue, we will have to add physics-informed constraints to our models, to ensure that physical laws are still being obeyed. For example, we must ensure that all contributions sum to one, we expect that the contributions will change smoothly over time, and that the dryness fraction will always be within the expected physical bounds of 0 and 1.

One way to do this is by using a Physics-Informed Neural Network (PINN). PINNs are able to incorporate these physical laws into their loss functions, where multiple functions can be minimised at once. The final loss function can become a sum of many individual loss functions relating to different physical laws, which are summed together with chosen weights which relate to the relative importance of those laws. Upon iteration through the network, the loss function will be minimised, thus specifying that the physical laws are being followed as they should. There are also loss functions applied for the accuracy of the data, and for the initial conditions and boundary conditions being met. PINNs generally work in discretised parameter spaces, which is appropriate to use in our context, because the data taken has been taken at discrete moments in time.

Another benefit of using a PINN is that it can model any physical scenario, as long it has an accompanying differential equation. There is no rule on how ‘complicated’ the equation can be and it is not required to have an analytical solution to be applied to a PINN. As geothermal physics can be represented by a highly complicated partial differential equation (PDE), this is ideal. It has also been used to model solutions in the field of fluid mechanics, and used on the notoriously difficult Navier-Stokes equations [24]. ML-augmented simulation has also been used by ANSYS, a premier fluid simulation tool, to “speed up the simulation by factors of 100x”, according to an ANSYS blog [25].

2.4 Solving as an Optimisation Problem

Another possible approach that could be taken is to solve this as an optimisation problem. As mentioned earlier, a Loss Function is a ...

mention uncertainty quantification

Chapter 3

Methodologies

3.1 Project Scope

The computational scope of this project is limited to the mass extraction data imputation and generation, which will then be provided to models created by Gravatt and O’Sullivan. The models using the data that we produce are out of scope for this project, as we are only focusing on the historical data prediction. The physical scope pertains to a geothermal field and its associated wells (real or simulated), which will provide a realistic amount of sparse data to our model. The models may be trained on data from many geothermal fields, but will model results for only one section of a geothermal field which feeds into a single separator (as shown in Figure 1.2) at a time.

Our data-driven models will take in two separate types of input data. Total steam flow measurements will be provided across all discretised time points, as well as sparse TFT (True Flow Test) measurements which will be provided only at a few time points across our time domain. Our model will output the mass extracted from each well at each of these discretised time points. We will be able to train and test our models on simulated geothermal field data, and will be able to use data augmentation to artificially increase the amount of training data that we have available.

Initially, I will reduce the project scope and simplify the problem by treating only mass flow as the variable of interest, however this is not an accurate assumption, because in reality dryness is not constant, and therefore a distinction must be made between the steam flow measurements taken at the separator, and the mass flow results that we are outputting. Later during model development, I will relax this assumption to create a model that more can more accurately represent the complex real-world phenomena involved in a geothermal system

3.2 Variable Definition

To effectively model the situation described above, we have chosen to set our main target (aka response) variables as the contribution from a certain well at a certain discretised point in time. Often, overall steam flow measurements are provided monthly, so we will be discretising our time domain into monthly segments. These will be labeled $t \in 1, 2, \dots, T$. In our spatial domain, there will be a finite amount of wells that will be labeled well 1 to well n , and will each have a contribution for each month that that well was actively turned on. So across our spatial and time dimensions, we have a 2 dimensional matrix w of well mass flows, shown in equation (3.2.1).

$$w \in \mathbb{R}^{n \times T} \tag{3.2.1}$$

where:

$w_{i,t}$ = mass extracted from well i at time t .

We will begin our project by attempting to solve a simplified version of the problem, treating all wells as having a constant average extraction rate. For these cases where I treat the mass extracted per well as constant over time, we set $w_{i,t} = w_i$, turning w from a 2 dimensional matrix as shown in equation (3.2.1) into a 1 dimensional vector. and assuming that we have consistent and accurate data for the total steam flow of the combined wells. We will also assume that we have at least one TFT measurement per well, to give us a starting point of what their mean contribution may be. Over time, we plan to relax or remove these simplifications, and add additional constraints to better capture the behaviour of the geothermal system.

3.3 Work Strategy

Myself and my project partner Michael Terekhin will each attempt to build a model that effectively solves this problem. I will attempt to use optimisation to find the theoretically optimal solution for the historical data prediction. I will begin with a naive model which assumes a constant extraction from each well, and assumes known dryness across the system (so that we can treat mass flows as the only relevant unit) and over time relax these assumption to build a model that has more flexibility and can represent the real results better. I will then conduct parameter tuning to try and optimise this model for given example data sets. I will also conduct uncertainty quantification for my model, to ensure that those using the data I generate can understand and use the variance of my predictions, so that they can correctly calculate the variance of their own predictions.

Meanwhile my project partner Michael Terekhin will be attempting to solve this problem using Gaussian Process Regression. He will also start with a simpler model, and slowly build its complexity until it can fully account for the processes occurring in a given geothermal system. Once Michael and I have both completed our models, we will compare the results that are given by both of our model, and see which of the models is more accurate. We will also investigate whether a joint model that combines results from both of our models would be superior to either individual model.

3.4 Tools

I will use a number of tools to solve this complex problem. I have chosen Python as the programming language I will use to develop a model. This is because Python is a language I am very familiar with and it has a number of helpful existing features and libraries which I will explain in further detail below. To reformulate it as an analytical model, I will use a symbolic mathematics library 'SymPy' to conduct the majority of analytical operations that I will need, such as rearranging, substituting, taking coefficients, and analytically differentiating. In conjunction with this, I will use Numpy, an extremely helpful and common Python library for numerical manipulation, as well as Matplotlib, which is an excellent library for producing and displaying high quality images and graphs. This will be very important as I will need to visualise my models' accuracy against chosen example data.

Furthermore, I will conduct my optimisation using a Jupyter notebook. Jupyter notebook is a web tool that allows me to create an interactive code document, including live code, equations, graphs, and more. It allows me to run only certain parts of my code at once, as opposed to traditional programming scripts which would always run the entire program. This allows me to reduce the computation cost of developing my optimisation models, and also to more easily visualise the results of my models.

During this project, I soon realised that my laptop had insufficient computation power to effectively solve large and difficult problems. To counteract this issue, I began using an online Jupyter notebook interface, called the Jupyter Notebook Service provided by ARDC Nectar Research Cloud, which is an Australian cloud computing service specifically designed to enable large-scale computing research for academics across Australia and New Zealand. This allowed me to do more computationally intensive operations, however

cite

because of my status as an undergraduate student, I was only entitled to a small and relatively weak virtual machine. This still suited my needs well, and allowed me to reliably conduct my modelling from any location. This site also came with a number of existing environments. I chose to use the data-science python environment, as this included Sympy, Matplotlib, Numpy, and all other packages that I required.

Chapter 4

Mathematical Formulation

4.1 Formulating an Initial Model

To formulate an initial model we begin with the idea of a loss function. A loss function is a function which takes in a set of decision variables, and evaluates how well these map to data by evaluating the associated 'cost' or 'error'. Different machine learning algorithms use different methods to minimise the loss function.

To formulate the loss function we sum the squared error measured against each of the two data sources. In equation (4.1.1) we see the total squared error of the sum of all predicted mass flows measured against the total measured mass flow. Meanwhile in equation (4.1.2) we see the squared error of our predicted mass flow rates against the known TFT values, for all timepoints when this is known. These equations assume a constant average mass extraction rate, and known dryness throughout the system.

$$S1 = \sum_{t=1}^T (M_t - \sum_{i=1}^n z_{i,t} w_i)^2 \quad (4.1.1)$$

$$S2 = \sum_{i=1}^n \sum_{t=1}^T d_{i,t} (m_{i,t} - w_i)^2 \quad (4.1.2)$$

where:

w_i = Average extraction mass flow rate of well i

M_t = Total extraction mass flow rate recorded between all wells at time t

$m_{i,t}$ = Extraction mass flow rate of well i at time t

$z_{i,t}$ = 1 if well i was on at time t, 0 otherwise.

$d_{i,t}$ = 1 if well i's flow rate was measured at time t, 0 otherwise.

T = Total number of discrete timepoints

n = Total number of wells in our system

These two expressions for the squared error measured against our two known sets of data can then be summed together, and weighted with a regularisation parameter, in the form of equation (4.1.3). This gives us the final expression for our initial loss function, shown in (4.1.4).

$$\mathcal{L}(w_1, w_2 \dots w_n) = S1 + \lambda S2 \quad (4.1.3)$$

where:

$\mathcal{L}(w_1, w_2 \dots w_n)$ = Total error value of loss function

λ = Regularisation parameter that controls the relative weight of error against our bulk

and TFT measurements.

$$\mathcal{L}(w_1, w_2 \dots w_n) = \sum_{t=1}^T (M_t - \sum_{i=1}^n z_{i,t} w_i)^2 + \lambda \sum_{i=1}^n \sum_{t=1}^T d_{i,t} (m_{i,t} - w_i)^2 \quad (4.1.4)$$

4.2 Solving The Model Analytically

Initially, I solved equation (4.1.4) by using a built-in Python SciPy optimisation solver, which did give good results but took an extremely long time to converge. It was clear that expanding the problem in any way would push this method of solving the problem into infeasibility. This lead me to investigate new methods of solving the system, and after discussion with my supervisors, we determined that differentiating analytically and setting this expression to 0 would be a much more efficient way to solve the problem. From now on I will solve the system by minimising the loss function analytically using this approach. I take the derivatives with respect to each w_i and set each of these to 0, as shown in equation (4.2.1).

$$\frac{\partial \mathcal{L}(w_1, w_2 \dots w_n)}{\partial w_i} = 0 \quad \forall i \in [1, n] \quad (4.2.1)$$

Because our original Loss function deals with squared errors, these derivatives are at most linear in each w_i . This means that by taking all n of these equations, subtracting the constant terms to the right hand side, and reformatting the n equations into matrix form, we find the following relationship, shown in (4.2.2). Therefore the optimal w vector can be found by evaluating this expression, either by evaluating $A^{-1}b$ or other equivalent technique built into Sympy's `solve()` function.

$$Aw = b \quad (4.2.2)$$

where:

w = n -long vector of optimal w_i values

$A \in \mathbb{R}^{n \times n}$

$A_{i,j}$ = j 'th coefficient of $\frac{\partial \mathcal{L}(w_1, w_2 \dots w_n)}{\partial w_i}$

$b \in \mathbb{R}^n$

b_i = constant term of $-\frac{\partial \mathcal{L}(w_1, w_2 \dots w_n)}{\partial w_i}$

In equation (4.1.4) the parameter λ represents a weighting of the importance given to the sum squared error against the TFT measurements relative to the importance given to the sum squared error against the bulk measurements. To find an appropriate value, I decided to give the two terms an 'equal' weighting by setting λ to a value that gives the two terms an equal expected error value. Equations (4.2.3) and (4.2.4) shows how we can derive the value for λ which will give an approximately equal error contribution for each of these terms, beginning with the observation that error within the bulk term will be approximately n times that of the error within an individual well measurements, because bulk values are n times larger, as they are made from the contributions of n wells.

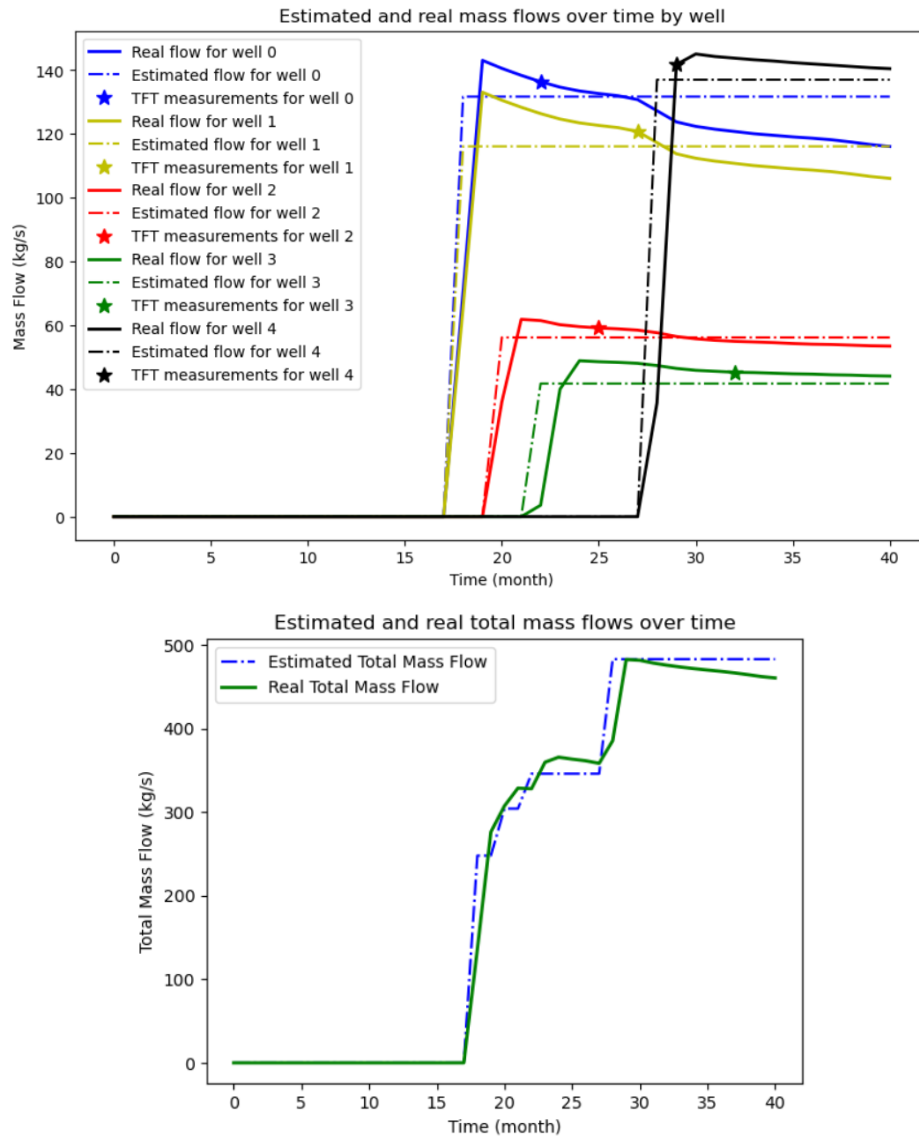


Figure 4.1: Mass flows predicted by initial model against real results for individual wells and total flows.

$$\begin{aligned}
E_{Bulk} &\approx n \times E_{TFT} \\
(E_{Bulk})^2 &\approx n^2 \times (E_{TFT})^2 \\
\sum_{t=1}^T \sum_{i=1}^n d_{i,t} (E_{TFT})^2 &\approx d_T (E_{TFT})^2 \\
\sum_{t=1}^T (E_{Bulk})^2 &\approx T (E_{Bulk})^2
\end{aligned} \tag{4.2.3}$$

where:

Substituting results of equation (4.2.3) into (4.1.4) we get equation (4.2.4), which neatly expresses lambda for a 'balanced' weighting between TFT and bulk measurement.

do
this

$$\begin{aligned}
\mathcal{L} &= \lambda d_T (E_{TFT})^2 + T (E_{Bulk})^2 \\
\mathcal{L} &= \lambda d_T (E_{TFT})^2 + T n^2 (E_{TFT})^2 \\
\lambda d_T (E_{TFT})^2 &\approx T n^2 (E_{TFT})^2 \\
\lambda d_T &\approx T n^2 \\
\lambda &\approx \frac{T n^2}{d_T}
\end{aligned} \tag{4.2.4}$$

4.3 Relaxing Model Assumptions

Equation (4.1.4) is our initial loss function, however it does have some shortcomings, because it is designed whilst making a number of key assumptions. Firstly, it assumes that the contribution from each well is constant over time. This is clearly not true, and is a poor assumption, but it helps us to greatly simplify the expression to create an initial loss function. Next we will relax this assumption by introducing linear time-dependence into the system. Instead of setting $w_{i,t} = w_i$, we will write it of the form $w_{i,t} = w_{c,i} + w_{g,i}t$ where $w_{c,i}$ represents the y-intercept of the line representing the extraction rate over time of well i, and $w_{g,i}$ represents its gradient. The new loss function is shown in equation (4.3.1)

$$\mathcal{L}(w_1, w_2 \dots w_n) = \sum_{t=1}^T (M_t - \sum_{i=1}^n z_{i,t} (w_{c,i} + w_{g,i}t))^2 + \lambda \sum_{i=1}^n \sum_{t=1}^T d_{i,t} (m_{i,t} - (w_{c,i} + w_{g,i}t))^2 \tag{4.3.1}$$

This results in a similar matrix equation as in equation (??), however the w vector will be twice as long because there are now two coefficients (w_c and w_g) for each well, and therefore $A \in \mathbb{R}^{2n \times 2n}$. This can again be solved symbolically to get the following results shown in 4.2. These results look similar to the results of the initial model, however because of the linear time-dependence, they are able to more effectively match the given data. We can see that with two TFT measurements per well, the model accurately fits to these data points.

4.4 Applying Tikhonov Regularisation to Reduce Overfitting

jhj

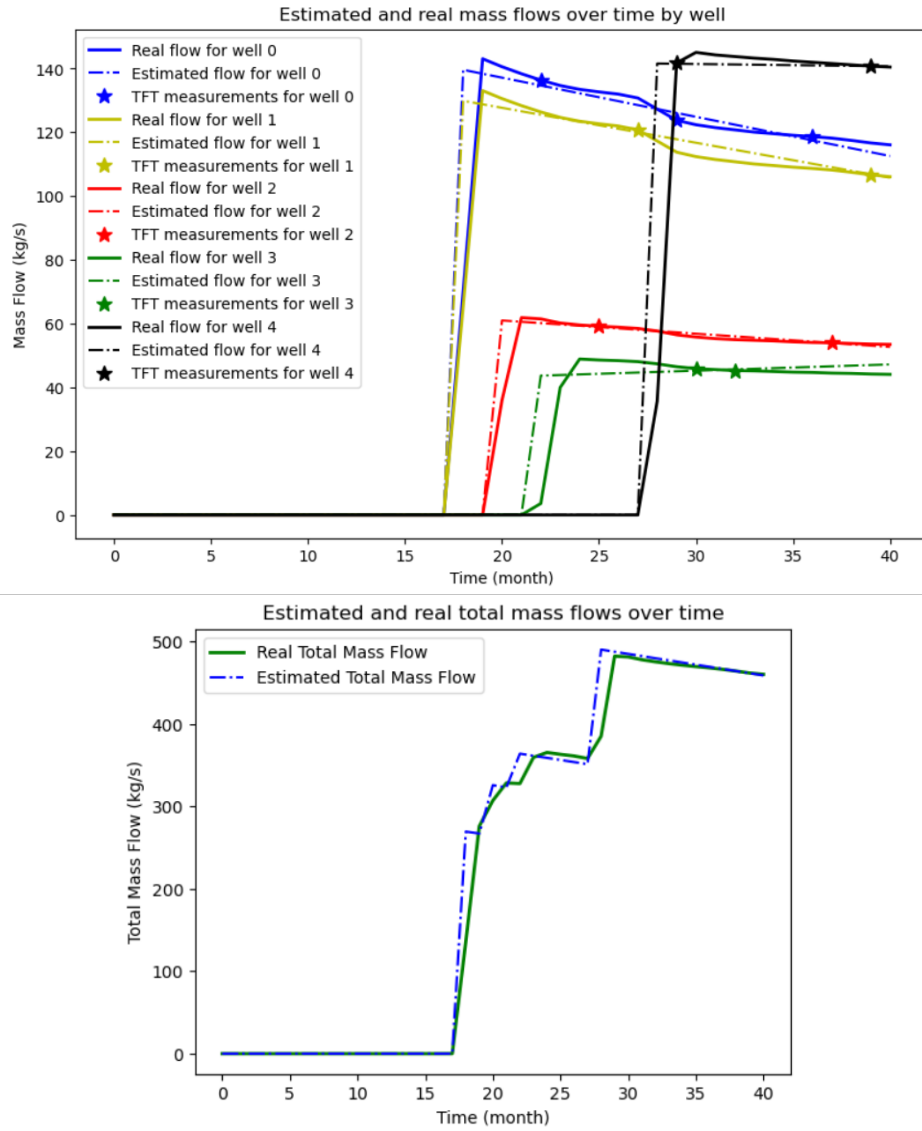


Figure 4.2: Mass flows predicted by linear model against real results for individual wells and total flows.

4.5 Solving Least Squares and Tuning Parameters

At this point I began to consider tuning the hyperparameters λ_i to make my model as effective as possible. To do this, I began by executing a logarithmic grid search using a number of possible values for each λ_i . This gave me an indication of which values to choose for λ_i , however I was unable to test each of these sets of values to an appropriate degree, as I was limited by the computational power available. To combat this, I decided to make my code more efficient by reformulating it as an overconstrained linear model which could then be solved using least squares very efficiently (CITE?).

I began with the general form of an inverse problem as shown in equation (4.5.1). Then to make the problem more digestible I split the A matrix into A_1 and A_2 , where A_1 contains other measured data such as if the wells were on/off, and A_2 contains the chosen model information, as shown in equation (4.5.2). Note that $k \ll T$, so the y vector is much longer than the x vector, meaning the problem is overconstrained. This is what allows us to solve it in a least squared fashion, to find the 'best fit' to the data by minimising squared error. A small example with arbitrary values of n , T and k is shown in equation (4.5.3)

$$\begin{aligned} y &= Ax + e \\ y &\approx Ax \end{aligned} \tag{4.5.1}$$

where:

$y \in \mathbb{R}^{T(n+1)}$ contains individual and total mass data.
 $A \in \mathbb{R}^{T(n+1) \times n(k+1)}$ contains model information and other measured data
 $x \in \mathbb{R}^{n(k+1)}$ is our vector of polynomial coefficients
 $e \in \mathbb{R}^{T(n+1)}$ is the error / noise

$$\begin{aligned} A &= A_1 A_2 \\ y &\approx A_1 A_2 x \\ y &\approx A_1 w \\ w &= A_2 x \end{aligned} \tag{4.5.2}$$

where:

$A_1 \in \mathbb{R}^{T(n+1) \times Tn}$ contains measurement and on/off well data
 $A_2 \in \mathbb{R}^{Tn \times n(k+1)}$ contains our chosen model
 $w \in \mathbb{R}^{Tn}$ is our vector of w values for all wells at all times

$$\begin{bmatrix} m_{11} \\ m_{12} \\ \vdots \\ m_{1T} \\ m_{21} \\ \vdots \\ m_{nT} \\ M_0 \\ M_1 \\ M_2 \\ \vdots \\ M_T \end{bmatrix} = \begin{bmatrix} d_{11} & 0 & \dots & 0 & 0 & \dots & \dots & 0 \\ 0 & d_{12} & & \vdots & \vdots & & & \vdots \\ \vdots & & \ddots & \vdots & \vdots & & & \vdots \\ 0 & \dots & \dots & d_{1T} & \vdots & & & \vdots \\ 0 & \dots & \dots & \dots & d_{21} & & & \vdots \\ \vdots & & & & & \ddots & & \vdots \\ \vdots & & & & & & \ddots & \vdots \\ 0 & \dots & \dots & \dots & \dots & \dots & \dots & d_{nT} \\ z_{11} & 0 & \dots & 0 & z_{21} & & & \vdots \\ 0 & z_{12} & & \vdots & & z_{22} & & \vdots \\ \vdots & & \ddots & \vdots & & & \ddots & \vdots \\ 0 & \dots & \dots & z_{1T} & & & & z_{nT} \end{bmatrix} \begin{bmatrix} 1 & t_1 & t_1^2 & \dots & t_1^k & 0 & \dots & \dots & \dots & 0 \\ 1 & t_2 & t_2^2 & \dots & t_2^k & \vdots & & & & \vdots \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & & & & \vdots \\ 1 & t_T & t_T^2 & \dots & t_T^k & 0 & \dots & \dots & \dots & 0 \\ 0 & \dots & \dots & \dots & 0 & 1 & t_1 & t_1^2 & \dots & t_1^k \\ \vdots & & & & \vdots & 1 & t_2 & t_2^2 & \dots & t_2^k \\ \vdots & & & & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & \dots & \dots & 0 & 1 & t_T & t_T^2 & \dots & t_T^k \end{bmatrix} \begin{bmatrix} x_{10} \\ x_{11} \\ x_{12} \\ \vdots \\ x_{1k} \\ x_{20} \\ x_{21} \\ x_{22} \\ \vdots \\ x_{nk} \end{bmatrix} \quad (4.5.3)$$

To check that I had done this correctly, I compared the results of solving the same system with the same provided data by both methods. Initially these results did not look the same. WHY?

This made the process of solving my model much more efficient. Upon comparison of the speed of each method, using the Python 'time' package, my old method took about 41 seconds to solve, meanwhile the new method took only about 3 seconds, indicating a speedup of around 1250%. so I was then able to execute much more extensive grid searches to find better results in a faster time.

Next, I applied Tikhonov regularisation to my new method of formulating and solving the problem. To do this I left my A_2 and x the same as before, but added additional rows to y and A_1 , which equates to adding more constraints. Because I can no longer differentiate analytically, the rows attached to A_1 will contain n finite-difference stencil blocks along the diagonal to numerically differentiate w . The first derivative version of this (for first order Tikhonov regularisation) is shown in equation (4.5.4), and the second order equivalent is shown in (4.5.5). Because we are trying to minimise the squared value of $\frac{dw}{dt}$ across time, we attach a 0 vector onto y .

$$A_{Tik,1} = \sqrt{\lambda_2} \begin{bmatrix} -1 & 1 & 0 & \dots & 0 & 0 & \dots & \dots & \dots & 0 \\ 0 & -1 & 1 & 0 & \vdots & \vdots & & & & \vdots \\ \vdots & & \ddots & \ddots & \vdots & \vdots & & & & \vdots \\ 0 & \dots & 0 & -1 & 1 & 0 & \dots & \dots & \dots & 0 \\ 0 & \dots & \dots & \dots & 0 & -1 & 1 & 0 & \dots & 0 \\ \vdots & & & & \vdots & 0 & -1 & 1 & 0 & \vdots \\ \vdots & & & & \vdots & & & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & \dots & 0 & 0 & \dots & 0 & -1 & 1 \end{bmatrix} \in \mathbb{R}^{n(T-1) \times nT} \quad (4.5.4)$$

$$A_{Tik,2} = \sqrt{\lambda_3} \begin{bmatrix} 1 & -2 & 1 & \dots & \dots & 0 & 0 & \dots & \dots & \dots & \dots & 0 \\ 0 & 1 & -2 & 1 & & \vdots & \vdots & & & & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & \vdots & \vdots & & & & & \vdots \\ 0 & \dots & \dots & 1 & -2 & 1 & 0 & \dots & \dots & \dots & \dots & 0 \\ 0 & \dots & \dots & \dots & \dots & 0 & 1 & -2 & 1 & \dots & \dots & 0 \\ \vdots & & & & & \vdots & 0 & 1 & -2 & 1 & & \vdots \\ \vdots & & & & & \vdots & & & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & \dots & \dots & 0 & 0 & \dots & 0 & 1 & -2 & 1 \end{bmatrix} \in \mathbb{R}^{n(T-2) \times nT} \quad (4.5.5)$$

These newly formed matrices $A_{Tikh,1}$ and $A_{Tikh,2}$ can be attached to our original A_1 matrix as shown in equation (4.5.6) to give us our new $A_{1,new}$ matrix that can be used in conjunction with y_{new} to solve in a least squares fashion using Tikhonov regularisation. Replacing y with y_{Tik} and A with A_{Tik} in equation (4.5.1) gives us our final least squares equation, shown in equation (4.5.7)

$$\begin{aligned} A_{1,Tik} &= \begin{bmatrix} A_1 \\ A_{Tik,1} \\ A_{Tik,2} \end{bmatrix} \in \mathbb{R}^{T(n+1)+n(T-1)+n(T-2) \times nT} \\ y_{Tik} &= \begin{bmatrix} y \\ 0 \\ 0 \end{bmatrix} \in \mathbb{R}^{T(n+1)+n(T-1)+n(T-2)} \\ A_{Tik} &= A_{1,Tik} A_2 \in \mathbb{R}^{T(n+1)+n(T-1)+n(T-2) \times n(k+1)} \end{aligned} \quad (4.5.6)$$

$$y_{Tik} \approx A_{Tik} x \quad (4.5.7)$$

Next I investigated the speedup of the new method against the old method of solving this problem. I define speedup as shown in equation (4.5.8). I evaluated this for a number of different values of T and k, to see if the speedup is dependent on these values. Results are shown in table (4.1). I do not currently possess the data required to test speedup for different n values, but we can safely assume that higher values exhibit large speedup, similar to large values of T and/or k. For all values examined, there is significant speedup. It is 12x faster for small problems (T=6,k=0), up to 1000x faster for large problems (T=41,k=5).

$$speedup[\%] = \frac{t_{old}}{t_{new}} * 100 \quad (4.5.8)$$

Interestingly, the only difference between the T=23 and T=41 examples are rows made entirely out of 0s, when all wells were turned off. While this does not provide any more information to our system, it does slow the system down in finding the optimal solution

Table 4.1: Speedup of new method compared to old method for different T and k values, with fixed n=5, rounded to nearest 100%.

$speedup[\%]$	k=0	k=1	k=2	k=3	k=4	k=5
T=6	1200	2400	6500	9700	17000	10400
T=11	2500	6000	10700	22700	28900	36800
T=17	3500	11100	22300	31800	45700	62500
T=23	7500	14900	26600	37800	55400	65800
T=41	21300	29600	43400	60600	79200	103500

(for both old and new methods). This means to increase efficiency even further I should remove all rows which are completely 0 in my y_{Tik} vector and A_{Tik} matrix, to further improve the efficiency of my system. Because only a small percentage of my m_{it} values are known (corresponding to points when $d_{it} = 1$), this will greatly reduce the size of the A array, making it theoretically faster to solve. At the small scales I am working at currently, there did not appear to be a significant speedup from making this change, because of the computational cost associated with doing these deletions. However for higher scale problems, this will be more efficient so I will leave this in. The results produced by the original and row-reduced methods are identical (up to machine error).

4.6 Quantifying Model Uncertainty

To Do .

Chapter 5

Results

5.1 Final Results

“I always thought something was fundamentally wrong with the universe”

5.2 Comparison with other models

5.3 Conclusions

References

- [1] J. H. S. Wendell A Duffield, Geothermal Energy, Clean Power from the Earth's Heat, U.S. Department of the Interior, 2003.
- [2] G. U.-b. Janusz A. Kubiak a, Simulation of the effect of scale deposition on a geothermal turbine, Geothermics (2002).
- [3] Nz geothermal fields, <https://www.geothermal.auckland.ac.nz/en.html> (2024).
- [4] L. Rybach, Geothermal sustainability, GHC Bulletin (2007).
- [5] D. Whiting, About us, <https://www.geothermal.auckland.ac.nz/en.html> (2024).
- [6] M. P. Hochstein, Assessment and modelling of geothermal reservoirs (small utilization schemes), Geothermics (1988).
- [7] R. van de Schoot et al., Bayesian statistics and modelling, Nature Reviews Methods Primers (2021).
- [8] J. K. C. Noah D. Athens, A monte carlo-based framework for assessing the value of information and development risk in geothermal exploration, Applied Energy (2019).
- [9] A. B. Downey, Think Bayes, Green Tea Press, 2012.
- [10] P. L. Bonate, A brief introduction to monte carlo simulation, Clinical Pharmacokinetics (2012).
- [11] T. G. Dietterich, Ensemble methods in machine learning, in: Multiple Classifier Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2000, pp. 1–15.
- [12] K. Csilléry, M. G. Blum, O. E. Gaggiotti, O. François, Approximate bayesian computation (abc) in practice, Trends in Ecology Evolution 25 (7) (2010) 410–418. doi:<https://doi.org/10.1016/j.tree.2010.04.001>.
URL <https://www.sciencedirect.com/science/article/pii/S0169534710000662>
- [13] M. Sadegh, J. A. Vrugt, Approximate bayesian computation using markov chain monte carlo simulation: Dream(abc), Water Resources Research 50 (8) (2014) 6767–6787. arXiv:<https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1002/2014WR015386>, doi:<https://doi.org/10.1002/2014WR015386>.
URL <https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1002/2014WR015386>
- [14] F. S. S. M. e. a. Barnes, C.P., Considerate approaches to constructing summary statistics for abc model selection, Stat Comput 22 (2012).
- [15] A. Bharti, Approximate bayesian computation with domain expert in the loop, <https://www.youtube.com/watch?v=Gx18PH0kiY8> (2023).
- [16] W. Q. Y. L. Xiuyong Shi, Degang Jiang, Application of the gaussian process regression method based on a combined kernel function in engine performance prediction, ACS Omega (2022).

- [17] M. I. Jordan, T. M. Mitchell, Machine learning: Trends, perspectives, and prospects, *Science* 349 (6245) (2015) 255–260. arXiv:<https://www.science.org/doi/pdf/10.1126/science.aaa8415>, doi:10.1126/science.aaa8415. URL <https://www.science.org/doi/abs/10.1126/science.aaa8415>
- [18] M. Y. Z. K. e. a. Wang, Q., A comprehensive survey of loss functions in machine learning, *Annals of Data Science* (2022).
- [19] E. R. O. et al., Machine learning in subsurface geothermal energy: Two decades in review, *Geothermics* (2022).
- [20] P. M. B. Abrasaldo, S. J. Zarrouk, A. W. Kempa-Liehr, A systematic review of data analytics applications in above-ground geothermal energy operations, *Renewable and Sustainable Energy Reviews* 189 (2024) 113998. doi:<https://doi.org/10.1016/j.rser.2023.113998>. URL <https://www.sciencedirect.com/science/article/pii/S1364032123008560>
- [21] M. R. Segal, Machine learning benchmarks and random forest regression, UCSF: Center for Bioinformatics and Molecular Biostatistics (2004).
- [22] J. Li, L. Qiu, B. Tang, D. Chen, D. Zhao, R. Yan, Insufficient data can also rock! learning to converse using smaller data with augmentation, *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01) (2019) 6698–6705. doi:10.1609/aaai.v33i01.33016698. URL <https://ojs.aaai.org/index.php/AAAI/article/view/4641>
- [23] I. W. K. S. M. S. Konstantinos Poulinakis, Dimitris Drikakis, Machine-learning methods on noisy and sparse data, *Mathematics* (2023).
- [24] M. Z. W. Z. e. a. Cai, S., Physics-informed neural networks (pinns) for fluid mechanics: a review, *Acta Mechanica Silica* (2021).
- [25] P. Banerjee, How ai and ml are changing simulation, *Ansys Blog* (2021).